

4. (4 points) Let $\vee, \wedge, \oplus$ denote respectively the OR, AND, and XOR Boolean logical operators, and $\neg$ denote Boolean negation. Assume Boolean values are represented as -1 (False) and $+1$ (True). Which of these logical functions cannot be learned by a single perceptron with inputs A and B ?

- $(A \wedge \neg B) \vee (\neg A \wedge B)$
- $(A \oplus B) \wedge A$
- $A \vee B$
- $\neg A \wedge B$

Solution: The answer is $(A \wedge \neg B) \vee (\neg A \wedge B)$, which is equal to $A \oplus B$.

5. (4 points) Let $L(\mathbf{w}) = \frac{1}{2} \sum_i (y_i - \mathbf{w}^\top \phi(x_i))^2$ be the loss function corresponding to a linear regression problem. Which equation represents the stochastic gradient descent update for $\mathbf{w}$?

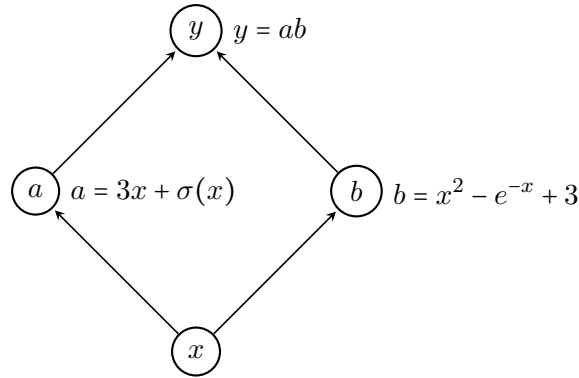
- $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta(y_i - \mathbf{w}^\top \phi(x_i))\phi(x_i)$
- $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \sum_i (y_i - \mathbf{w}^\top \phi(x_i))\phi(x_i)$
- $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta(y_i - \text{sign}(\mathbf{w}^\top \phi(x_i)))\phi(x_i)$, where $\text{sign}(\cdot)$ is the sign function
- $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta(y_i - \sigma(\mathbf{w}^\top \phi(x_i)))\phi(x_i)$, where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function.

Solution: Stochastic gradient updates depend only on a single example (or a mini-batch of examples). The option $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \sum_i (y_i - \mathbf{w}^\top \phi(x_i))\phi(x_i)$ corresponds to gradient descent on the full batch.

6. (4 points) Which one of the following statements is true?
- Convolutional layers are equivariant to translations and rotations.
 - Neural networks with a single hidden layer with linear activations are universal approximators.
 - Auto-encoders with non-linear activations and a squared loss are equivalent to PCA.
 - None of the above.

Solution: Convolutional layers are equivariant to translations, but not rotations. Neural networks with a single hidden layer with **linear** activations are equivalent to linear classifiers, which are not universal approximators. Auto-encoders with **linear** activations would correspond to PCA.

7. (4 points) Consider the following computation graph, where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function. What is the derivative of y with respect to x ?



- $b(3 + \sigma(x)(1 - \sigma(x))) + a(2x + e^{-x})$.
- $a(3 + \sigma(x)(1 - \sigma(x))) + b(2x + e^{-x})$.
- $3 + \sigma(x)(1 - \sigma(x)) + 2x + e^{-x}$.
- 0.

Solution: It is $b(3 + \sigma(x)(1 - \sigma(x))) + a(2x + e^{-x})$:

$$\begin{aligned}
 \frac{\partial y}{\partial a} &= b, & \frac{\partial y}{\partial b} &= a \\
 \frac{\partial a}{\partial x} &= 3 + \sigma(x)(1 - \sigma(x)), & \frac{\partial b}{\partial x} &= 2x + e^{-x} \\
 \frac{\partial y}{\partial x} &= \frac{\partial y}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial y}{\partial b} \frac{\partial b}{\partial x} = b(3 + \sigma(x)(1 - \sigma(x))) + a(2x + e^{-x}).
 \end{aligned}$$

8. (4 points) Which one of the following statements is **false**?
- **Gradient clipping can prevent vanishing gradients.**
 - Transformer models can be used for computer vision applications.
 - Distributed representations generally require fewer dimensions than local (one-hot) representations.
 - Upper level layers (closer to the output) tend to learn more abstract representations (shapes, forms, objects) compared to bottom level layers.

Solution: Gradient clipping can prevent exploding gradients, not vanishing gradients.

Part 2: Short Answer Questions (18 points)

Please provide **brief** answers (1-2 sentences) to the following questions.

1. (6 points) Explain how dropout regularization works.

Solution: At training time, for each example neurons are dropped randomly with probability p (i.e. their activations are masked to become zero) and the remaining activations are scaled by $1/(1 - p)$. This forces each neuron to depend less on other neurons' activations.

2. (6 points) Explain the role and need for positional encoding in transformers.

Solution: Without positional encodings, the self-attention in transformers is insensitive to the word positions being queried: permuting the words leads to a similar permutation in the self-attention responses. In order for transformers to be sensitive to the word order, each word embedding is augmented with a positional embedding.

- (6 points) Mention one advantage of contextualized word embeddings (e.g. BERT) over static word embeddings (e.g. word2vec or GloVe).

Solution: Contextualized word embeddings can assign different representations to the same word being used in different contexts; this is particularly useful for polysemic words (such as “bank” which can be a river bank or a financial institution).

Part 3: Problems (50 points)

Problem 1: Convolutional Neural Networks (25 points)

In their retail store, Yolanda and Zach currently use a card punching system to register the entry and exit times of their 6 employees. However, they heard about recent advances in computer vision systems and decided to replace that system by face recognition using CNNs.

To train the system, they collected a large dataset of pictures from their 6 employees, Alice, Berta, Chad, Diane, Eric, and Frank. Each picture in the dataset is a 192×256 grayscale picture, similar to those depicted in Fig. 1, and is labeled according to the corresponding employee.

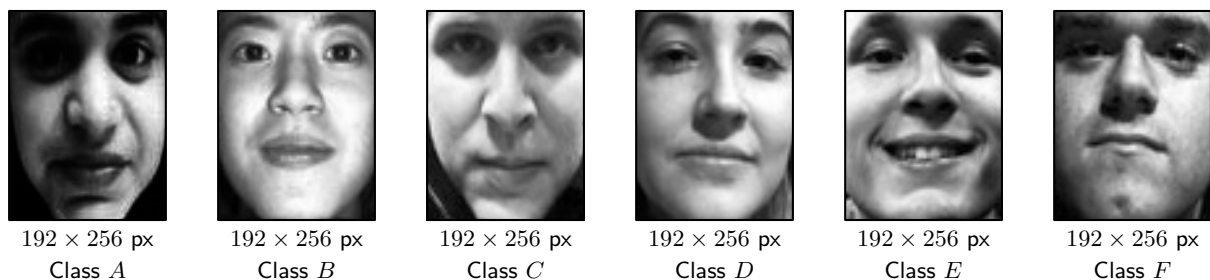
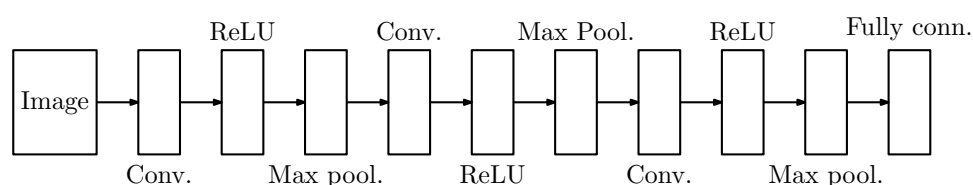


Figure 1: Sample pictures from the 6 classes that the CNN must recognize. Alice corresponds to class A , Berta to class B , etc.

- (4 points) Briefly explain in 1-2 sentences why a CNN is an adequate choice of architecture for Yolanda and Zach’s task (image classification).

Solution: CNNs take advantage of the spacial structure of the image, unlike standard feed-forward networks. Moreover, convolutional and pooling layers exploit the fact that the same feature may appear in different parts of the image, enabling the network to process those occurrences in a similarly way.

- (7 points) Suppose that, in their classifier, their use the following architecture:



which is specified using the following Pytorch code snippet:

```
nn.Sequential(
    nn.Conv2d(1, 5, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(5, 10, kernel_size=5, stride=1, padding=0),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(10, 20, kernel_size=2, stride=2, padding=0),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=5, stride=2),
    nn.Flatten(),
    nn.Linear(2800, 6))
```

Fill in the following table with the adequate values.

Layer	Output size	N. weights	N. biases
Input	$192 \times 256 \times 1$	0	0
1st conv. layer	$192 \times 256 \times 5$	45	5
1st pooling layer	$96 \times 128 \times 5$	0	0
2nd conv. layer	$92 \times 124 \times 10$	1250	10
2nd pooling layer	$46 \times 62 \times 10$	0	0
3rd conv. layer	$23 \times 31 \times 20$	800	20
3rd pooling layer	$10 \times 14 \times 20$	0	0
Output layer	6×1	16,800	6

3. (7 points) Consider the diagram in Fig. 2, containing the brightness values for the first window of pixels in one of the images in the dataset.

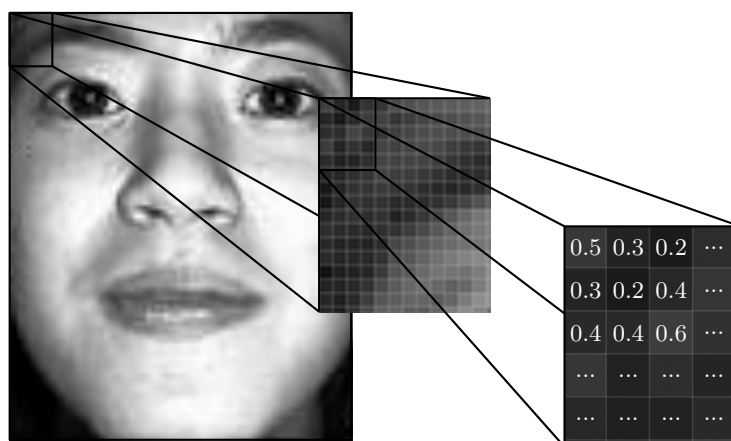


Figure 2: Brightness values for one of the images in the dataset.

Suppose that, after training, one of the filters in the first convolutional layer is defined by

the parameters

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad b = 0.5.$$

For the filter provided, compute the the top-left-most value after the pooling layer. Do not forget that the first convolutional layer includes a padding of size 1 (use zeros as the padding value).

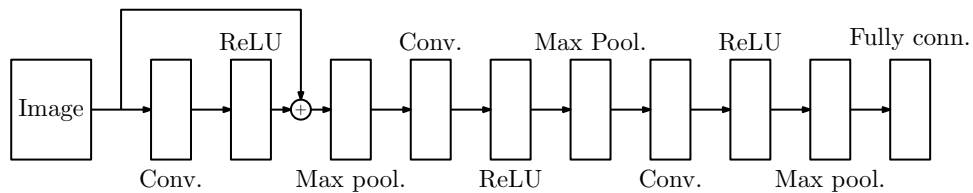
Solution: The input of the max-pool layer corresponding to the provided pixels is given by

$$\begin{aligned} \mathbf{h}_{\text{conv}} &= \text{ReLU} \left(\begin{bmatrix} -0.5 & 0.2 \\ -0.9 & 0.0 \end{bmatrix} + 0.5 \right) \\ &= \begin{bmatrix} 0.0 & 0.7 \\ 0.0 & 0.5 \end{bmatrix}. \end{aligned}$$

At the output of the pooling layer, we thus have

$$h_{\text{pool}} = 0.7.$$

4. (7 points) Suppose that Yolanda and Zach decide to add some skip connections to their network, as indicated in the diagram:



Repeat Question 3, but now considering the skip connection indicated in the diagram above.

Solution: The input of the first max-pool layer corresponding to the provided pixels is given by

$$\begin{aligned} \mathbf{h}_{\text{conv}} &= \text{ReLU} \left(\begin{bmatrix} -0.5 & 0.2 \\ -0.9 & 0.0 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.3 \\ 0.3 & 0.2 \end{bmatrix} + 0.5 \right) \\ &= \begin{bmatrix} 0.5 & 1.0 \\ 0.0 & 0.7 \end{bmatrix}. \end{aligned}$$

At the output of the pooling layer, we thus have

$$h_{\text{pool}} = 1.0.$$

Problem 2: Sequence-to-Sequence Models (25 points)

Bartholomew (known to his friends as Bart) had an idea for a project: building a system to summarize news articles into a short sentence (e.g., a tweet). He collected a dataset with news documents and their corresponding tweets, which he will use to train a summarization model.

1. (7 points) Bart's sister (Lisa) is taking a course on *deep learning* and she recommended using a sequence-to-sequence architecture based on a *recurrent neural network* (RNN) for this problem. Bart tested a simple RNN-based sequence-to-sequence model (without any attention mechanism) on a small-scale experiment. He is using a very small vocabulary (7 words, including the $\langle \text{STOP} \rangle$ symbol), shared between the source and target, and using the same embedding vectors for both sides. The embedding vectors are

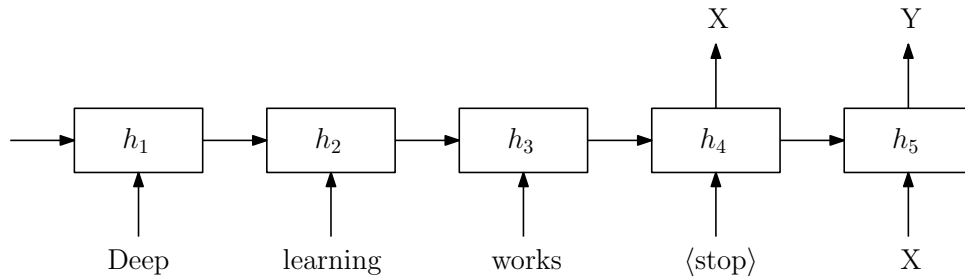
$$\begin{aligned} \mathbf{x}_{\text{Deep}} &= [0, 1]^\top, & \mathbf{x}_{\text{learning}} &= [1, 0]^\top, & \mathbf{x}_{\text{works}} &= [-1, -1]^\top, \\ \mathbf{x}_{!!!} &= [2, -1]^\top, & \mathbf{x}_{\# \text{deep}} &= [-1, 2]^\top, & \mathbf{x}_{\text{lol}} &= [0, -1]^\top, & \mathbf{x}_{\langle \text{stop} \rangle} &= [1, 1]^\top. \end{aligned}$$

The initial hidden state of the RNN, $\mathbf{h}_0$, is all-zeros. The input-to-hidden matrix is

$$\mathbf{W}_{hx} = \begin{bmatrix} 0 & -1 \\ 2 & 0 \\ 1 & 1 \end{bmatrix}.$$

The recurrent matrix $\mathbf{W}_{hh}$ is the identity matrix. All biases are vectors of zeros. The RNN uses **relu** activations.

Compute the last state of the encoder RNN, $\mathbf{h}_4$, for the input document “Deep learning works”. Show all your calculations.



Solution: We have:

$$\begin{aligned} \mathbf{h}_1 &= \text{relu}(\mathbf{W}_{hx}\mathbf{x}_{\text{Deep}} + \mathbf{W}_{hh}\mathbf{h}_0) \\ &= \text{relu}([-1, 0, 1]^\top + [0, 0, 0]^\top) \\ &= [0, 0, 1]^\top. \\ \mathbf{h}_2 &= \text{relu}(\mathbf{W}_{hx}\mathbf{x}_{\text{learning}} + \mathbf{W}_{hh}\mathbf{h}_1) \\ &= \text{relu}([0, 2, 1]^\top + [0, 0, 1]^\top) \\ &= [0, 2, 2]^\top. \\ \mathbf{h}_3 &= \text{relu}(\mathbf{W}_{hx}\mathbf{x}_{\text{works}} + \mathbf{W}_{hh}\mathbf{h}_2) \\ &= \text{relu}([1, -2, -2]^\top + [0, 2, 2]^\top) \\ &= [1, 0, 0]^\top. \\ \mathbf{h}_4 &= \text{relu}(\mathbf{W}_{hx}\mathbf{x}_{\langle \text{stop} \rangle} + \mathbf{W}_{hh}\mathbf{h}_3) \\ &= \text{relu}([-1, 2, 2]^\top + [1, 0, 0]^\top) \\ &= [0, 2, 2]^\top. \end{aligned}$$

Therefore the last state is $\mathbf{h}_4 = [0, 2, 2]^\top$.

2. (8 points) Assume that in the previous question we obtained $\mathbf{h}_4 = [0, 2, 2]^\top$. Assume that the decoder RNN has the same parameters as the encoder RNN, and the hidden-to-output matrix is

$$\mathbf{W}_{yh} = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 1 & -1 \\ -2 & 1 & 0 \\ 0 & 0 & -1 \\ 1 & 2 & 0 \\ 0 & 0 & 0 \\ -2 & 0 & 1 \end{bmatrix}.$$

The target word probabilities at time step t are given by $\text{softmax}(\mathbf{W}_{yh}\mathbf{h}_t)$, where $\mathbf{h}_t$ is the corresponding state of the decoder RNN.

Compute the first two words of the generated tweet using **greedy decoding**.

Solution: We have:

$$\begin{aligned} \mathbf{y}_1 &= \text{argmax}(\mathbf{W}_{yh}\mathbf{h}_4) \\ &= \text{argmax}([-2, 0, 2, -2, 4, 0, 2]) = \text{\#deep}. \\ \mathbf{h}_5 &= \text{relu}(\mathbf{W}_{hx}\mathbf{x}_{\text{\#deep}} + \mathbf{W}_{hh}\mathbf{h}_4) \\ &= \text{relu}([-2, -2, 1]^\top + [0, 2, 2]^\top) \\ &= [0, 0, 3]^\top. \\ \mathbf{y}_2 &= \text{argmax}(\mathbf{W}_{yh}\mathbf{h}_5) \\ &= \text{argmax}([0, -3, 0, -3, 0, 0, 3]) = \text{<stop>}. \end{aligned}$$

The generated words are “\#deep <stop>”.

3. (6 points) After playing with this network for a while, Bart realized that it didn't work well for long documents and therefore decided to add an attention mechanism. In the first decoding step, using **scaled dot-product attention** with $\mathbf{h}_4$ as the query vector and $\mathbf{h}_1$, $\mathbf{h}_2$, $\mathbf{h}_3$ as the key and value vectors, compute the attention probabilities and the resulting context vector (use $\mathbf{h}_1 = [0, 0, 1]^\top$, $\mathbf{h}_2 = [0, 2, 2]^\top$, $\mathbf{h}_3 = [1, 0, 0]^\top$, $\mathbf{h}_4 = [0, 2, 2]^\top$).

Solution: We have:

$$\begin{aligned} s_1 &= \frac{1}{\sqrt{3}}[0, 2, 2]^\top [0, 0, 1] = \frac{2}{\sqrt{3}} \\ s_2 &= \frac{1}{\sqrt{3}}[0, 2, 2]^\top [0, 2, 2] = \frac{8}{\sqrt{3}} \\ s_3 &= \frac{1}{\sqrt{3}}[0, 2, 2]^\top [1, 0, 0] = 0 \\ Z &= \exp\left(\frac{2}{\sqrt{3}}\right) + \exp\left(\frac{8}{\sqrt{3}}\right) + \exp(0) = 105.546 \\ \mathbf{p} &= \text{softmax}([s_1, s_2, s_3]) = \exp([s_1, s_2, s_3])/Z = [.030, .960, .009] \\ \mathbf{c} &= \mathbf{h}_1 p_1 + \mathbf{h}_2 p_2 + \mathbf{h}_3 p_3 = [.009, 1.921, 1.951]. \end{aligned}$$

4. (4 points) Lisa's friend, Allison, who is also knowledgeable about deep learning, told Bart about transformers and large pretrained models. Give one example of a pretrained model that Bart could use for this task and the necessary steps to use it.

Solution: Bart could use a pretrained decoder-only (e.g., GPT) or encoder-decoder model (e.g., T5, BART) and fine-tune it on the data he has available. Alternatively he could use the model without any fine-tuning and use prompting at test time. A possible prompt would be "<document> TL;DR: <answer>". Note: an encoder-only model (e.g. BERT) would not be suitable, since this an auto-regressive generation task.